

Guía Definitiva: El Ciclo de Aprendizaje SAC bajo el Capó

(Referencia Completa para el Proyecto de Portafolios DRL con SAC)

Esta guía detalla el flujo exacto de datos, las transformaciones matemáticas y la configuración técnica que ocurren dentro de la librería `stable_baselines3` (SB3) al entrenar un agente SAC (Soft Actor-Critic). SAC es un algoritmo **Off-Policy** y de **Máxima Entropía**.

1. LA ARQUITECTURA: EL ECOSISTEMA DE 5 REDES

A diferencia de PPO, SAC requiere un ecosistema de 5 redes neuronales trabajando en paralelo:

1. **Actor (Política):** Decide qué acciones tomar.
2. **Crítico Gemelo 1 (Q-Network 1):** Evalúa financieramente la decisión del Actor.
3. **Crítico Gemelo 2 (Q-Network 2):** Hace una segunda evaluación independiente.
4. **Target Crítico 1:** Copia rezagada del Crítico 1 (para estabilidad).
5. **Target Crítico 2:** Copia rezagada del Crítico 2 (para estabilidad).

2. FASE DE INTERACCIÓN Y MEMORIA

Paso 1: La Observación (Input de la Red)

Al inicio de cada periodo t , el entorno ensambla el **Vector de Estado** (o_t) y se lo entrega al Actor. Este vector contiene: Capital actual (x_t), pesos anteriores (w_{t-1}), el tiempo (t/T) y las *features* del mercado (medias, varianzas, covarianzas).

Paso 2: La Decisión "Blanda" (Política Estocástica)

El estado pasa por las neuronas del **Actor**.

- La red arroja los parámetros de una campana de Gauss (Media μ y Desviación Estándar σ).
- El algoritmo hace un muestreo aleatorio ("tira los dados") dentro de esa campana para elegir una acción continua $\tilde{a}_t$.
- Tu entorno aplica la fórmula de proyección para convertir ese número en porcentajes de inversión válidos (pesos $w_{i,t}$) que sumen 100%.

Paso 3: Consecuencias y Recompensa (Step)

El entorno avanza una semana con esos pesos:

1. Calcula los retornos descontando los **costos de transacción** (c_{SPX}, c_{CMC}).
2. Actualiza tu capital a x_{t+1} .

3. Calcula la **Recompensa Financiera** (r_t).

Paso 4: El Archivo Histórico (Replay Buffer)

El agente empaqueta la experiencia (s_t, a_t, r_t, s_{t+1}) y la guarda en el **Replay Buffer**.

- `buffer_size` : Es el límite de esta memoria (por defecto 1,000,000 de pasos).
- **Lógica FIFO**: Como SAC no olvida al terminar un episodio, este buffer se va llenando de semanas históricas. Si se alcanza el millón, borra las memorias más viejas para hacer espacio.

3. FASE DE APRENDIZAJE: TIEMPOS Y OPTIMIZACIÓN

Como SAC es *Off-Policy*, la recolección de datos y el aprendizaje están separados. No usa el `n_steps` de PPO.

Paso 5: La Pausa para Estudiar (`train_freq`)

El código usa el hiperparámetro `train_freq` para saber cuándo pausar el juego y aprender. Si `train_freq=1` , el agente pausa la simulación **después de cada paso (semana)**.

Paso 6: Muestreo del Pasado (Mini-Batches)

Al pausar, el agente mete la mano en el Replay Buffer y saca un "puñado" aleatorio (Mini-batch) de experiencias pasadas. Puede mezclar la Semana 14 del Episodio 2 con la Semana 90 del Episodio 50.

Paso 7: Los Críticos Gemelos y el Pesimismo

Por cada experiencia sacada del buffer, el Actor propone qué haría *hoy* en esa situación. Luego, los Críticos estiman el valor $Q(s, a)$: "*¿Cuánto dinero total ganaremos si tomamos esta acción?*".

- **El Truco del Pesimismo**: El Crítico 1 predice \$500. El Crítico 2 predice \$450. SAC **siempre toma el valor más bajo** (\$450). Esto evita que la IA se vuelva peligrosamente optimista e invierta todo en predicciones infladas.

Paso 8: Backpropagation y la Función Loss (El Núcleo del Aprendizaje)

Aquí entra el optimizador **Adam**. Su tamaño de "paso" matemático se define con el hiperparámetro `learning_rate` . Adam ajusta las redes intentando minimizar el error (Loss):

1. **Loss de los Críticos**: Comparan su predicción pesimista contra lo que *realmente* se ganó en ese recuerdo del Replay Buffer. Aplican *backpropagation* para reducir su Error Cuadrático Medio, volviéndose mejores evaluadores.

2. **Loss del Actor (El objetivo principal):** Adam ajusta los pesos neuronales del Actor buscando maximizar la siguiente ecuación combinada:

$$\text{Objetivo} = \text{Predicción Pesimista de Ganancias } Q(s, a) + (\alpha \times \text{Entropía})$$

- **La Temperatura (α / `ent_coef`):** Si α es alto, el Actor ganará más puntos por mantener su campana de Gauss ancha (ser impredecible/explorar) que por ganar unos pocos dólares en el mercado. Esto asegura que no se estanque en estrategias mediocres.

Paso 9: Repetición de Gradientes (`gradient_steps`)

El hiperparámetro `gradient_steps` le dice a Adam cuántas veces debe repetir los Pasos 6, 7 y 8 antes de "despausar" la simulación. Si es igual a 1, hace un solo ajuste de neuronas y vuelve a jugar.

Paso 10: El Suavizado de las Target Networks (τ)

Si los Críticos principales cambian radicalmente tras el *backpropagation*, el Actor se marearía. Para evitarlo, las **Target Networks** (usadas para calcular el error) se actualizan usando el parámetro **Tau** (τ). Si $\tau = 0.005$, la Target Network toma solo un 0.5% de los nuevos conocimientos de los Críticos principales y mantiene un 99.5% de sus valores antiguos. Este suavizado hace que el aprendizaje sea extremadamente estable frente a la volatilidad financiera.

Resumen del Ciclo Físico: El agente da un paso → Guarda en Replay Buffer → Si se cumple `train_freq`, pausa → Saca mini-batches aleatorios → Evalúa pesimistamente → Adam ajusta neuronas con *Backprop* priorizando (Ganancia+Entropía) repitiendo esto `gradient_steps` veces → Suaviza redes objetivo con τ → Reanuda el juego.